ORBITL1-AX25

PHASE 4 TECHNICAL REPORT | RELIABILITY, ACCEPTANCE & PERFORMANCE BENCHMARK

Phase 4 COMPLETE	95 / 95 Tests PASS	864.0 Frames / s	0 Defects / Regressions
Software Acceptance Gate Status	Cumulative Verification Suite	Sustained Processing Rate	Runtime Code Baseline Integrity

Phase 4 Mission & Acceptance Objectives

Phase 4 establishes the formal software reliability and acceptance baseline for the ORBITL-1 AX.25 protocol implementation. Building upon Phase 1 (Frame Core), Phase 2 (HDLC/NRZI), and Phase 3 (E2E Integration), Phase 4 validates system robustness under multi-seed randomized testing, pathological byte distributions, strict protocol boundary enforcement, and mixed continuous fault streams containing interleaved CRC corruptions, HDLC abort sequences, and oversized frames.

Cumulative Test Suite Composition by Responsibility

Test Module	Test Class Focus	Tests	Phase Responsibility	Pass Rate
tests/test_ax25_core.py	Address encoding/decoding, UI frame builder, FCS validation, profile edge cases	29	Phase 1: AX.25 Frame Core	100% (29/29)
tests/test_hdlc.py	Bit stuffing, NRZI transitions, chunked HDLC decoding, bit validation	10	Phase 2: HDLC & NRZI	100% (10/10)
tests/test_receiver.py	AX25ReceivePipeline, NRZI demodulator handoff, streaming chunks	7	Phase 2: Receive Pipeline	100% (7/7)
tests/test_reliability.py	Automatic resynchronization, noise recovery, size boundary regression, 1000 rounds	18	Phase 2/4: Robustness Baseline	100% (18/18)
tests/test_integration.py	End-to-end integration, arbitrary chunk sizes, fault recovery, spec vectors, reference model	22	Phase 3: Integration & Interoperability	100% (22/22)
tests/test_acceptance.py	Multi-seed testing, profile rejection (Control/PID), boundary enforcement, stress benchmark	9	Phase 4: Reliability & Acceptance	100% (9/9)
TOTAL CUMULATIVE	Full Automated Test Suite across all phases (executed via standard unittest)	95	Phases 1 through 4 Combined	100% (95/95)

Phase 4 Reliability Methodology & Verification Evidence

Detailed results of randomized testing, pathological inputs, and strict profile rejection.

1. Deterministic Multi-Seed Randomized Testing

To ensure deterministic and reproducible verification, testing utilizes fixed pseudorandom seeds (20261009, 20261019, 42, 1337). For each seed, 25 randomized payloads ranging from 0 to 1,024 bytes were encoded into AX.25 UI frames, framed via HDLC, modulated to NRZI levels, and streamed into the receive pipeline. 100% of packets were recovered with bit-for-bit accuracy and valid FCS verification.

2. Pathological Payload Survival Matrix

Test Pattern	Data Size	Stress Factor	Bit Stuffing Behavior	Receiver Result
All 0x00 bytes	256 bytes	Dense 0-bits (maximum NRZI line transitions)	Zero bit stuffing required; maximum transition rate	PASS (100% match)
All 0xFF bytes	256 bytes	Dense 1-bits (maximum bit stuffing stress)	Stuffs zero bit after every 5 ones; unstuffs cleanly	PASS (100% match)
Alternating 0x55 / 0xAA	256 bytes each	High-frequency bit inversion (01010101 and 10101010)	Tests byte boundary alignment and shift registers	PASS (100% match)
Ascending / Descending Ramps	512 bytes each	Covers all 256 octet values sequentially in both directions	Ensures no byte-value sensitivity in parser	PASS (100% match)
HDLC Flag Octets (0x7E)	128 bytes	Accidental flag sequence (01111110) in payload data	Bit stuffing successfully prevents false frame close	PASS (100% match)
HDLC Abort Octets (0x7F)	128 bytes	Accidental abort sequence (01111111) in payload data	Bit stuffing breaks consecutive 1s; no abort triggered	PASS (100% match)

3. Strict Profile Rejection Matrix

Rejected Profile Parameter	Tested Values	Enforcement Point	Observed System Response
Non-UI Control Fields	0x00, 0x10 (I-frame), 0x01 (RR), 0x05 (RNR), 0x09 (REJ), 0x2F (SABM), 0x43 (DISC)	_decode_ui_body() in src/ax25.py	ValueError('Unsupported AX.25 control field'); receiver drops frame
Non-0xF0 Protocol IDs	0x01 (X.25), 0x08 (Q.931), 0x81 (CDPD), 0xCC (IPv4), 0xFF, 0x00	_decode_ui_body() in src/ax25.py	ValueError('Unsupported AX.25 PID'); receiver drops frame
Malformed Wire Addresses	Destination extension bit=1; Source extension bit=0; clear reserved bits	_decode_ui_body() and decode_address()	ValueError('Destination address must not have final-address bit set')
Oversized Information Field	4079 bytes payload (exceeds 4078-byte profile cap)	build_ui_frame() and HDLC size guard	build_ui_frame raises ValueError; receiver drops wire frame cleanly

Throughput Benchmarks & Implementation Evidence

Execution metrics and verified source excerpts from the production codebase.

Throughput Benchmark Summary (test_sustained_load_throughput_benchmark)

A sustained stress workload of 500 frames (74,000 total bytes) was streamed through the complete NRZI + HDLC + AX.25 pipeline. The software receiver processed the entire batch in 0.579 seconds, achieving a sustained throughput of 864.0 frames/second (1,022.98 kbps / ~1.02 Mbps). Compared to typical amateur satellite telemetry rates of 1.2 kbps or 9.6 kbps, the software protocol stack provides over 100x processing headroom.

Production Source Code Excerpts (Verified Truth)

```
# From src/ax25.py: build_ui_frame
def build_ui_frame(destination: str, source: str, information: bytes) -> bytes:
    information = _require_bytes(information, "information")
    if len(information) > MAX_INFORMATION_BYTES:  # 4078 bytes
        raise ValueError(f"Information field exceeds {MAX_INFORMATION_BYTES} bytes...")
    body = bytearray()
    body.extend(encode_address(destination, last=False))
    body.extend(encode_address(source, last=True))
    body.extend((AX25_UI_CONTROL, AX25_NO_LAYER3_PID))  # 0x03, 0xF0
    body.extend(information)
    fcs = crc16_x25(bytes(body))
    body.extend((fcs & 0xFF, (fcs >> 8) & 0xFF))  # low byte first
    return bytes((AX25_FLAG,)) + bytes(body) + bytes((AX25_FLAG,))
```

```
# From src/ax25_receiver.py: feed_nrzi_levels
def feed_nrzi_levels(self, levels, initial_level: int | None = None):
    levels = list(levels)
    if initial_level is not None:
        self._nrzi_previous_level = initial_level
    if not levels: return []
    bits = nrzi_decode(levels, initial_level=self._nrzi_previous_level)
    self._nrzi_previous_level = levels[-1]  # Retain state across streaming chunks
    return self.feed_bits(bits)
```

```
# From src/vendor/mobilinkd/HDLC.py: _finish_frame and process
def _finish_frame(self):
    length = self.frame.tell()
    if 14 < length <= self.MAX_FRAME_BYTES:  # 4096 bytes cap
        return (self.crc(), self.frame.getvalue()[:-2])
    return None
```

Phase 4 Hardware Demarcation & Software Acceptance

Independent vector validation, hardware demarcation, and actionable HIL roadmap.

Hardware-in-the-Loop (HIL) Preparation Checklist

Subsystem / Milestone	Prerequisite Completed in SW	Hardware Requirement	Verification Target
AFSK 1200 Baud Audio	NRZI & bitstream verified	Bell 202 audio DAC/ADC interface	Zero CRC errors on loopback
GFSK 9600 Baud SDR	4096B stream buffer guard	SDR transceiver (HackRF/USRP/Lime)	Demodulator handoff to feed_nrzi_levels
Doppler Tracking	Chunked streaming state	Gpredict / Hamlib frequency steering	Carrier lock maintained through ±3.5 kHz shift
Telecommand Auth	FCS validation & error drop	HMAC / AES crypto wrapper on payload	Anti-replay protection on uplink telecommands

Formal Software Acceptance Declaration

PHASE 4 & CUMULATIVE PROJECT ACCEPTANCE GATE: MET (SOFTWARE-COMPLETE)
All 95 unit, integration, reliability, and acceptance tests pass deterministically across all 4 phases.
Runtime code integrity of Phases 1 through 4 has been preserved without unauthorized modification.
The software protocol layer is fully verified, documented, and verified under sustained workload benchmark (864 frames/s).
Physical RF hardware qualification remains TBR pending radio hardware selection and HIL testing in Phase 5.